Unit 1

1.1 Introduction to PHP

PHP (Hypertext Preprocessor) is a widely-used open-source scripting language especially suited for web development. It is embedded within HTML and used to manage dynamic content, databases, session tracking, and even build entire e-commerce sites.

1.1.1 Understanding the Role of PHP in Server-side Web Development

◈ What is PHP?

- PHP stands for **PHP: Hypertext Preprocessor**.
- It is an **open-source**, **server-side scripting language** designed for web development.
- Embedded within HTML code and executed on the server.

◈ Server-side Scripting:

- PHP runs on the **server**, not the client (browser).
- Generates **dynamic web content** based on user input, session state, database data, etc.
- For example, login forms, registration systems, blogs, content management systems (CMS), etc.

◈ How PHP Works:

1. User requests a .php page through browser (e.g., http://localhost/home.php).
2. Apache (web server) forwards the request to the PHP interpreter.
3. PHP processes the code (e.g., interacts with database).
4. Generates HTML as output and sends it to the browser.

◈ Advantages of PHP:

- Cross-platform (Windows, Linux, macOS).
- Integrates with many databases (MySQL, PostgreSQL, Oracle, SQLite).
- Supports session management, cookies, file uploads, and more.
- Large community and library support (e.g., Composer, PEAR).

1.1.2 History and Evolution of PHP

◈ Brief History:

Year	Version	Key Features
1994	PHP Tools	Created by Rasmus Lerdorf for personal use

Year Version	Key Features
1995 PHP/FI	Form Interpreter, support for HTML forms
1998 PHP 3	True scripting language, increased functionality
2000 PHP 4	Powered by Zend Engine 1.0, better performance
2004 PHP 5	Object-Oriented Programming (OOP), MySQLi, PDO
2015 PHP 7	Huge performance improvement, strict typing
2020 PHP 8	JIT compiler, attributes, union types

🔹 **Key Milestones:**

- **Zend Engine**: Core scripting engine behind PHP.
- **PHP 5**: Introduced full OOP support (classes, objects, constructors, inheritance).
- **PHP 7 & 8**: Improved performance and introduced modern programming concepts.

1.1.3 Installation and Configuration Using XAMPP/WAMP

To run PHP on your local machine, you can use a package like XAMPP or WAMP:

- XAMPP: Cross-platform (Windows, Linux, Mac). Includes Apache (web server), MySQL (database), PHP, and Perl.
- WAMP: Windows-specific. Includes Windows, Apache, MySQL, and PHP.

Steps to Install XAMPP:

1. Download from https://www.apachefriends.org/
2. Run the installer and follow the setup wizard.
3. Start Apache and MySQL from the XAMPP Control Panel.
4. Place your PHP files in the htdocs folder (e.g., C:\xampp\htdocs\project\index.php).
5. Open browser and type http://localhost/project/index.php.

Basic Configuration:

- Change php.ini settings to enable/disable extensions (like mysqli, curl, etc.).
- Configure Apache's httpd.conf to change port, document root, etc.

1.1.4 Setting up the Development Environment Using Visual Studio Code (VS Code)

VS Code is a lightweight and powerful source code editor with support for PHP development.

Steps to Set Up VS Code for PHP:

1. Install VS Code from https://code.visualstudio.com/.
2. Install PHP Extension:

- Open VS Code → Go to Extensions → Search "PHP Intelephense" → Install.

3. Configure PHP Executable Path (optional):
   - Open settings.json and set path to PHP (e.g., C:/xampp/php/php.exe).
4. Install Additional Extensions (Optional but Recommended):
   - PHP Debug
   - Code Runner
   - Prettier (for formatting)
5. Run PHP Code:
   - Save file with .php extension in htdocs folder.
   - Run using browser at http://localhost/filename.php.

📌 **Example: First PHP Program**

File: first.php
Location: C:\wamp\www\first.php

```
<?php
echo "Hello, World! This is my first PHP program.";
?>
```

Run in Browser:
http://localhost/hello.php

1.2 Basic PHP Syntax and Variables

1.2.1 PHP Script Structure and Tags

Introduction:
PHP (Hypertext Preprocessor) is a server-side scripting language used to develop dynamic and interactive web pages. PHP scripts are executed on the server, and the result is sent to the browser as plain HTML.

Basic Structure:
Every PHP script starts with <?php and ends with ?>.

```
<?php
   // PHP code goes here
?>
```

Example:

```
<!DOCTYPE html>
```

```
<html>
<head>
   <title>My First PHP Page</title>
</head>
<body>
   <?php
      echo "Hello, World!";
   ?>
</body>
</html>
```

Key Points:

- PHP code can be embedded in HTML.
- The file extension should be .php.
- Statements end with a semicolon ;.

Alternative Tag Styles (Not recommended unless specifically enabled in php.ini):

```
<script language="php"> echo "Hello"; </script>
<% echo "Hello"; %>  // ASP-style tags
```

1.2.2 Declaring and Using Variables and Constants

Variables:

- Variables in PHP are declared using a dollar sign ($) followed by the variable name.
- Variable names are case-sensitive.
- PHP is a loosely typed language, so you do not need to declare a data type.

Syntax:

```
$variable_name = value;
```

Example:

```
$name = "John";
$age = 25;
$price = 49.99;
```

Rules for Variables:

- Must start with a letter or underscore (_)
- Cannot start with a number
- Can contain letters, numbers, and underscores

Constants:

- Constants are similar to variables but their value cannot be changed once defined.
- They are defined using define() or const.

Syntax:

define("SITE_NAME", "MyWebsite");

const AUTHOR = "Admin";

Example:

define("PI", 3.14);

echo PI;

Difference between Variables and Constants:

Feature	Variable	Constant
Changeable	Yes	No
Syntax	$name	define() or const
Scope	Function or Global	Global only
Prefix required	Yes ($)	No

1.2.3 Using echo and print Statements

echo Statement:

- Used to output one or more strings.
- Faster than print and does not return any value.

Syntax:

echo "Hello World!";

echo "Name: ", $name, " Age: ", $age;

print Statement:

- Outputs a string.
- Slower than echo but returns 1, so it can be used in expressions.

Syntax:

print "Welcome to PHP!";

Differences:

Feature	Echo	Print
Return Value	No	Yes (1)
Multiple Params	Yes	No
Speed	Slightly faster	Slightly slower

Example:

$name = "Poonam";

echo "Hello ", $name;

print "Welcome to the PHP tutorial.";

1.2.4 Comments and Formatting Conventions

Comments in PHP:
Used to add notes or explanations in code. They are ignored during execution.

Types of Comments:

1. **Single-line comment**

// This is a single-line comment

This is also a single-line comment

2. **Multi-line comment**

/*

This is a multi-line comment

It spans more than one line

*/

Best Practices for Commenting:

- Use comments to explain complex logic.
- Avoid redundant comments.
- Keep comments up-to-date with the code changes.

Formatting Conventions:

Practice	Example
Indentation	Use 4 spaces or a tab for nested blocks
Naming variables	Use descriptive names: $studentName, $totalPrice
Code readability	Keep code clean and well-organized
Spacing	Add space around operators: $a + $b

Example of Well-Formatted Code:

```
<?php
    // Calculate total price
    $price = 100;
    $tax = 0.05;
    $total = $price + ($price * $tax);
    echo "Total amount: " . $total;
?>
```

Summary

- PHP scripts use <?php ?> tags and can be embedded in HTML.
- Variables are defined with $, while constants use define() or const.
- Use echo or print to display output.
- Commenting is essential for maintainability and understanding code.
- Formatting conventions ensure readable and maintainable code.

1.3 Data Types and Operators

PHP is a loosely typed language, which means you don't have to declare a variable's data type explicitly. PHP automatically converts the variable to the correct data type depending on its value.

1.3.1 Primitive Types

✅ String

A string is a sequence of characters, enclosed in **single (')** or **double ("")** quotes.

```
$name = "Poonam";
echo "Hello, $name"; // Output: Hello, Poonam
```

✅ Integer (int)

An integer is a whole number without a decimal point.

```
$age = 25;
echo $age; // Output: 25
```

✅ **Float (or double)**

A float is a number with a decimal point or in exponential form.

```
$price = 99.99;
echo $price; // Output: 99.99
```

✅ **Boolean**

A boolean represents two possible states: **TRUE** or **FALSE**.

```
$is_admin = true;
if ($is_admin) {
    echo "Access granted"; // Output: Access granted
}
```

1.3.2 Arrays and Objects

✅ **Arrays**

Arrays are collections of values. They can be indexed or associative.

Indexed Array:

```
$fruits = array("Apple", "Banana", "Mango");
echo $fruits[1]; // Output: Banana
```

Associative Array:

```
$student = array("name" => "Raj", "age" => 20);
echo $student["name"]; // Output: Raj
```

✅ **Objects**

Objects are instances of classes. Used in object-oriented programming.

```
class Car {
    public $color = "Red";
    function drive() {
        echo "Driving...";
    }
}
```

```
$myCar = new Car();
echo $myCar->color; // Output: Red
$myCar->drive();    // Output: Driving...
```

1.3.3 Type Casting and Type Juggling

✅ Type Casting

Type Casting is the process where you manually change the type of a variable.

In PHP you can do casting as follows:

- Cast to (int) or (integer) → Integer
- Cast to (bool) or (boolean) → Boolean
- Cast to (float) or (double) or (real) → Float
- (string) → Cast to String
- (array) → Cast to Array
- (object) → Cast to Object
- (unset) → cast to NULL

Examples of Type Casting

Casting String to Integer

```
<?php
   $x = "100";   //String
   $y = (int)$x; // String cast to Integer.
   var_dump($y); // output: int(100)
?>
```

Casting Float to Integer

```
<?php
   $x = 10.75;   //Float
   $y = (int)$x; // Float cast to Integer.
   echo $y;      // output: 10
?>
```

Casting Integer to Boolean

```
<?php
   $x = 0;       //Integer
```

```
$y = (bool)$x; // Integer cast to Boolean.
var_dump($y); // output: bool(false)
?>
```

Casting to Array

```
<?php
$x = "Hello";
$y = (array)$x; // String cast to Array.
print_r($y);    // Output: Array ( [0] => Hello )
?>
```

✅ Type Juggling

PHP is a loosely typed language, which means that it is not necessary to explicitly define the data type of variables when declaring them.

PHP can change the data type automatically (type juggling) or you can change it manually (type casting).

Type juggling occurs when PHP automatically changes the data type depending on the situation where the variable is being used.

Example: automatic type conversion

```
<?php
$x = "10";  //String
$y = 20;    //Integer
$result = $x + $y;  // PHP converts "10" to Integer.
echo $result;       // output: 30
?>
```

Things to keep in mind while type juggling:

1. PHP looks at the relevant context when converting types.
2. Tries to mix types of variables while using operators.

Example: Type Juggling with Logical Operators

```
<?php
$a = "0";      //String
$b = 0;        //Integer
var_dump($a == $b);  // true because both "0" and 0 are considered equal.
```

```
var_dump($a === $b); // false because the types are different (String and Integer).
?>
```

Difference Between Type Juggling and Type Casting

Type Juggling	Type Casting
The process is done automatically by PHP.	Done manually by the developer.
The developer has less control.	The developer has complete control.
Example $x = “10” + 20;	$x = (int)”10”;

Common problems related to Type Juggling

1. **Unintended Consequences:**

```
<?php
   $x = “10 apples”;
   $y = 5;
   echo $x + $y;  // Output: 15 (“10” converted to Integer, “apples” ignored.)
?>
```

Need for strict comparison:

When it is necessary to check both the value and the type, use === (strict comparison).

```
<?php
   $a = “0”;  //String
   $b = 0;    //Integer
   var_dump($a === $b);  // Output: false (because the types are different.)
?>
```

Correct use of Type Casting

1. **Explicitly cast the data:**

Use casting when you need to convert a variable to a certain type.

2. **Check the data:**

Before casting ensure that the data is of the correct type and has no errors.

Conclusion

- PHP’s Type Juggling is flexible and useful, but it can sometimes be confusing.
- Using Type Casting you can get more control and make your code error-free

1.3.4 Operators

Operators are used to perform operations on variables and values.

PHP divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Increment/Decrement operators
- Logical operators
- String operators
- Array operators
- Conditional assignment operators

PHP Arithmetic Operators

The PHP arithmetic operators are used with numeric values to perform common arithmetical operations, such as addition, subtraction, multiplication etc.

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y
**	Exponentiation	$x ** $y	Result of raising $x to the $y'th power

PHP Assignment Operators

The PHP assignment operators are used with numeric values to write a value to a variable.

The basic assignment operator in PHP is "=". It means that the left operand gets set to the value of the assignment expression on the right.

Assignment	Same as...	Description
x = y	x = y	The left operand gets set to the value of the expression on the right
x += y	x = x + y	Addition
x -= y	x = x – y	Subtraction
x *= y	x = x * y	Multiplication
x /= y	x = x / y	Division
x %= y	x = x % y	Modulus

PHP Comparison Operators

The PHP comparison operators are used to compare two values (number or string):

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y

===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y
<=>	Spaceship	$x <=> $y	Returns an integer less than, equal to, or greater than zero, depending on if $x is less than, equal to, or greater than $y. Introduced in PHP 7.

PHP Increment / Decrement Operators

The PHP increment operators are used to increment a variable's value.

The PHP decrement operators are used to decrement a variable's value.

Operator	Name	Description
++$x	Pre-increment	Increments $x by one, then returns $x
$x++	Post-increment	Returns $x, then increments $x by one
--$x	Pre-decrement	Decrements $x by one, then returns $x
$x--	Post-decrement	Returns $x, then decrements $x by one

PHP Logical Operators

The PHP logical operators are used to combine conditional statements.

Operator	Name	Example	Result
And	And	$x and $y	True if both $x and $y are true
Or	Or	$x or $y	True if either $x or $y is true
Xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true

!	Not	!$x	True if $x is not true

PHP String Operators

PHP has two operators that are specially designed for strings.

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

PHP Array Operators

The PHP array operators are used to compare arrays.

Operator	Name	Example	Result
+	Union	$x + $y	Union of $x and $y
==	Equality	$x == $y	Returns true if $x and $y have the same key/value pairs
===	Identity	$x === $y	Returns true if $x and $y have the same key/value pairs in the same order and of the same types

!=	Inequality	$x != $y	Returns true if $x is not equal to $y
<>	Inequality	$x <> $y	Returns true if $x is not equal to $y
!==	Non-identity	$x !== $y	Returns true if $x is not identical to $y

PHP Conditional Assignment Operators

The PHP conditional assignment operators are used to set a value depending on conditions:

Operator	Name	Example	Result
?:	Ternary	$x = *expr1* ? *expr2* : *expr3*	Returns the value of $x. The value of $x is *expr2* if *expr1* = TRUE. The value of $x is *expr3* if *expr1* = FALSE
??	Null coalescing	$x = *expr1* ?? *expr2*	Returns the value of $x. The value of $x is *expr1* if *expr1* exists, and is not NULL. If *expr1* does not exist, or is NULL, the value of $x is *expr2*. Introduced in PHP 7

1.4 Control Structures and Arrays

1.4.1 Conditional statements: if, else, elseif, switch

Very often when you write code, you want to perform different actions for different conditions. You can use conditional statements in your code to do this.

In PHP we have the following conditional statements:

- if statement - executes some code if one condition is true
- if...else statement - executes some code if a condition is true and another code if that condition is false
- if...elseif...else statement - executes different codes for more than two conditions
- switch statement - selects one of many blocks of code to be executed

PHP - The if Statement

The if statement executes some code if one condition is true.

Syntax

```
if (condition)
 {
  code to be executed if condition is true;
 }
```

Example

Output "Have a good day!" if the current time (HOUR) is less than 20:

```
<?php
$t = date("H");

if ($t < "20") {
  echo "Have a good day!";
}
?>
```

PHP - The if...else Statement

The if...else statement executes some code if a condition is true and another code if that condition is false.

Syntax

```
if (condition)

{
  code to be executed if condition is true;
} else {
  code to be executed if condition is false;
}
```

Example

Output "Have a good day!" if the current time is less than 20, and "Have a good night!" otherwise:

```
<?php
$t = date("H");
```

```
if ($t < "20") {
  echo "Have a good day!";
} else {
  echo "Have a good night!";
}
?>
```

PHP - The if...elseif...else Statement

The if...elseif...else statement executes different codes for more than two conditions.

Syntax

```
if (condition)
{
  code to be executed if this condition is true;
}
elseif (condition)
{
  code to be executed if first condition is false and this condition is true;
}
else
{
  code to be executed if all conditions are false;
}
```

Example

Output "Have a good morning!" if the current time is less than 10, and "Have a good day!" if the current time is less than 20. Otherwise it will output "Have a good night!":

```
<?php
$t = date("H");

if ($t < "10")
{
  echo "Have a good morning!";
}
elseif ($t < "20")
{
  echo "Have a good day!";
}
else
```

```
{
  echo "Have a good night!";
}
?>
```

PHP switch Statement

Use the switch statement to **select one of many blocks of code to be executed**.

Syntax

```
switch (n)

{
  case label1:
    code to be executed if n=label1;
    break;
  case label2:
    code to be executed if n=label2;
    break;
  case label3:
    code to be executed if n=label3;
    break;
    ...
  default:
    code to be executed if n is different from all labels;
}
```

This is how it works: First we have a single expression *n* (most often a variable), that is evaluated once.

The value of the expression is then compared with the values for each case in the structure.

If there is a match, the block of code associated with that case is executed.

Use break to prevent the code from running into the next case automatically.
The default statement is used if no match is found.

Example

```
<?php
$favcolor = "red";

switch ($favcolor)

{
  case "red":
    echo "Your favorite color is red!";
    break;
  case "blue":
```

```
    echo "Your favorite color is blue!";
    break;
  case "green":
    echo "Your favorite color is green!";
    break;
  default:
    echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

1.4.2 Looping constructs: for, while, do-while, foreach

In PHP, Loops are used to repeat a block of code multiple times based on a given condition. PHP provides several types of loops to handle different scenarios, including while loops, for loops, do...while loops, and foreach loops.

In this article, we will discuss the different types of loops in PHP, their syntax, and examples.

Types of Loops in PHP

Below are the following types of loops in PHP:

- for loop
- while loop
- do-while loop
- foreach loop

🔁 1. for Loop

Best use case: When you know in advance how many times you want to run a block of code.

Syntax:

```
for (initialization; condition; increment/decrement) {
   // block of code
}
```

Explanation:

- **Initialization:** Sets the loop counter (e.g., $i = 0)
- **Condition:** Loop continues **while condition is true**
- **Increment:** Updates the counter ($i++ increases it by 1)

Example:

```
for ($i = 1; $i <= 5; $i++) {
```

```
    echo "Number: $i <br>";
}
```

Output:

Number: 1

Number: 2

Number: 3

Number: 4

Number: 5

🔁 2. while Loop

Best use case: When you **don't know in advance** how many times the loop should run.

Syntax:

```
while (condition) {
    // block of code
}
```

Example:

```
$i = 1;
while ($i <= 3) {
    echo "Value is $i <br>";
    $i++;
}
```

Output:

Value is 1

Value is 2

Value is 3

🔁 3. do-while Loop

Best use case: Ensures the loop **runs at least once**, even if the condition is false.

Syntax:

```
do {
```

```
    // block of code
} while (condition);
```

Example:

```
$i = 10;
do {
    echo "This runs once even if condition fails. <br>";
} while ($i < 5);
```

Output:

This runs once even if condition fails.

🔁 4. foreach Loop

Best use case: Used to **iterate over arrays** easily.

Syntax:

```
foreach ($array as $value)
{
    // code
}
```

Example with values:

```
$colors = array("Red", "Green", "Blue");
foreach ($colors as $value) {
    echo "$ value <br>";
}
```

Output:

Red

Green

Blue

Example with key-value pairs (associative array):

```
$student = array("Name" => "Raj", "Age" => 20);
foreach ($student as $key => $value) {
   echo "$key: $value <br>";
}
```

Output:

Name: Raj

Age: 20

✅ 1.4.3 Arrays in PHP

📘 1. Indexed Array

- Indexes are numeric (0-based)
- Created using array() or short syntax []

Example:

```
$fruits = ["Apple", "Banana", "Mango"];
echo $fruits[0]; // Output: Apple
```

Loop through indexed array:

```
for ($i = 0; $i < count($fruits); $i++) {
   echo $fruits[$i] . "<br>";
}
```

📘 2. Associative Array

- Uses **named keys** instead of numeric indexes.

Example:

```
$person = array("name" => "Amit", "age" => 30);
echo $person["name"]; // Output: Amit
```

Loop through associative array:

```
foreach ($person as $key => $value) {
   echo "$key = $value <br>";
}
```

📓 3. Multidimensional Array

- Array containing **other arrays**.
- Can be indexed, associative, or both.

Example:

```
$students = array(
   "Raj" => array("Math" => 80, "Science" => 70),
   "Simran" => array("Math" => 85, "Science" => 90),
   "Megha" => array("English" => 85, "SS" => 90));
```

Name	Math	Science	English	SS
Raj	80	70		
Simran	85	90		
Megha			85	90

```
echo $students["Raj"]["Science"]; // Output: 70
```

Nested Loop Example:

```
foreach ($students as $name => $subjects)
{
        echo "$name's marks:<br>";
        foreach ($subjects as $subject => $mark)
        {
                echo "$subject: $mark <br>";
        }
        echo "<br>";
}
```

✅ 1.4.4 Array Operations

🔧 1. sort() – Sorts indexed array in ascending order

```
$numbers = [4, 2, 8, 1];
sort($numbers);
print_r($numbers);
```

Output:

Array ([0] => 1 [1] => 2 [2] => 4 [3] => 8)

🔧 **2. asort() – Sorts associative array by value, maintains key association.**

```
$marks = array("Raj" => 80, "Simran" => 80, "Amit" => 100);
asort($marks);
print_r($marks);
```

Output:

Array ([Amit] => 70 [Raj] => 80 [Simran] => 90)

🔧 **3. ksort() – Sorts associative array by key.**

```
$marks = array("Simran" => 90, "Raj" => 80, "Amit" => 70);
ksort($marks);
print_r($marks);
```

Output:

Array ([Amit] => 70 [Raj] => 80 [Simran] => 90)

🔧 **4. array_merge() – Combines two or more arrays into one.**

```
$arr1 = array("a", "b");
$arr2 = array("c", "d");
$combined = array_merge($arr1, $arr2);
print_r($combined);
```

Output:

Array ([0] => a [1] => b [2] => c [3] => d)

1.5 Functions and Form Handling

🔷 1.5.1 Creating and Invoking User-Defined Functions

✅ **Definition:**

Functions allow you to group code into reusable blocks. PHP allows you to define your own functions using the function keyword.

✅ **Syntax:**

```
function functionName()
```

```
{
   // code block
}
```

✅ **Example:**

```
function greet()
{
   echo "Welcome to PHP!";
}
greet(); // Function call
```

Output:

```
Welcome to PHP!
```

🔷 1.5.2 Function Parameters and Return Values

✅ **Parameters:**

You can pass values (arguments) into a function.

✅ **Return Values:**

A function can return a value using the return keyword.

✅ **Example with Parameters:**

```
function add($a, $b)
{
   return $a + $b;
}

$result = add(5, 10);
echo "Sum: $result";
```

Output:

```
Sum: 15
```

🔷 1.5.3 Variable Scope: Global vs. Local

✅ **Local Variables:**

Declared inside a function and accessible **only within** that function.

✅ **Global Variables:**

Declared **outside** of functions and accessed inside using the global keyword.

✅ **Example:**

```
$x = 10;
$y = 20;

function sum()
{
  // global $x, $y; // accessing global variables
   $z = $x + $y;
   echo "Sum = $z";
}
Echo $z;
sum();
```

Output:

Sum = 30

✅ **Local Example:**

```
function localExample()
{
   $a = 100; // local to this function
   echo "Local variable: $a";
}
localExample();
// echo $a; // Error: $a is not defined outside the function
```

🔷 1.5.4 Handling Forms with $_GET and $_POST

Forms are used to collect data from users. The data is sent to the server using GET or POST methods.

✅ GET Method:

- Appends data to the URL.
- Suitable for small and non-sensitive data.
- Data visible in URL.

HTML Form:

```
<form method="get">
   Name: <input type="text" name="username">
   <input type="submit" value="Submit">
</form>
```

PHP Script:

```
if (isset($_GET['username'])) {
   $name = $_GET['username'];
   echo "Hello, $name";
}
```

URL Example:
If you enter **Raj**, the URL becomes:

yourfile.php?username=Raj

Output:

Hello, Raj

✅ POST Method:

- Sends data in HTTP request body (not visible in URL).
- Suitable for large or sensitive data.

HTML Form:

```
<form method="post">
   Email: <input type="email" name="email">
   <input type="submit" value="Submit">
</form>
```

PHP Script:

```
if (isset($_POST['email']))
 {
   $email = $_POST['email'];
   echo "Your Email: $email";
}
```

🔷 1.5.5 Basic Input Validation and Sanitization

Validation and sanitization are critical to ensure **security** and **data integrity**.

✅ 1. Validation

Check if required input is given and in correct format.

```
if (empty($_POST['name']))
{
        echo "Name is required.";
}
```

✅ 2. Sanitization

Remove unwanted characters (e.g., HTML tags).

```
$name = $_POST['name'];
$name = trim($name);              // Remove extra spaces
$name = stripslashes($name);      // Remove slashes
$name = htmlspecialchars($name);  // Convert special characters
```

✅ Full Example: Basic Form Handling with Validation

HTML + PHP:

```
<form method="post">
   Name: <input type="text" name="name"><br>
   <input type="submit">
</form>
```

```php
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST")
{
   if (empty($_POST["name"]))
  {
      echo "Name is required!";
   }
  else
 {
      $name = htmlspecialchars(trim($_POST["name"]));
      echo "Welcome, $name";
   }
}
?>
```

Output (if user enters "Poonam"):

Welcome, Poonam

Summary:

Concept	Description
User-defined function	Custom reusable code block
Parameters & return	Input/output values of a function
Scope	Variable accessibility (local/global)
$_GET	Retrieves form data via URL
$_POST	Retrieves form data securely
Validation & Sanitization	Makes input safe and usable

Validating data = Determine if the data is in proper form.

Sanitizing data = Remove any illegal character from the data.

The PHP Filter Extension

PHP filters are used to validate and sanitize external input.

The PHP filter extension has many of the functions needed for checking user input, and is designed to make data validation easier and quicker.

The filter_list() function can be used to list what the PHP filter extension offers:

ExampleGet your own PHP Server

```
<table>
 <tr>
  <td>Filter Name</td>
  <td>Filter ID</td>
 </tr>
 <?php
 foreach (filter_list() as $id =>$filter) {
  echo '<tr><td>' . $filter . '</td><td>' . filter_id($filter) . '</td></tr>';
 }
 ?>
</table>
```

Why Use Filters?

Many web applications receive external input. External input/data can be:

- User input from a form
- Cookies
- Web services data
- Server variables
- Database query results

You should always validate external data!

Invalid submitted data can lead to security problems and break your webpage!
By using PHP filters you can be sure your application gets the correct input!

PHP filter_var() Function

The filter_var() function both validate and sanitize data.

The filter_var() function filters a single variable with a specified filter. It takes two pieces of data:

- The variable you want to check

- The type of check to use

Sanitize a String

The following example uses the filter_var() function to remove all HTML tags from a string:

Example

```
<?php
$str = "<h1>Hello World!</h1>";
$newstr = filter_var($str, FILTER_SANITIZE_STRING);
echo $newstr;
?>
```

Validate an Integer

The following example uses the filter_var() function to check if the variable $int is an integer. If $int is an integer, the output of the code below will be: "Integer is valid". If $int is not an integer, the output will be: "Integer is not valid":

Example

```
<?php
$int = 100;

if (!filter_var($int, FILTER_VALIDATE_INT) === false)
{
        echo("Integer is valid");
}
else
{
         echo("Integer is not valid");
}
?>
```

Tip: filter_var() and Problem With 0

In the example above, if $int was set to 0, the function above will return "Integer is not valid". To solve this problem, use the code below:

Example

```
<?php
$int = 0;

if (filter_var($int, FILTER_VALIDATE_INT) === 0 || !filter_var($int,
```

```
FILTER_VALIDATE_INT) === false) {
  echo("Integer is valid");
} else {
  echo("Integer is not valid");
}
?>
```

Validate an IP Address

The following example uses the filter_var() function to check if the variable $ip is a valid IP address:

Example

```
<?php
$ip = "127.0.0.1";

if (!filter_var($ip, FILTER_VALIDATE_IP) === false) {
  echo("$ip is a valid IP address");
} else {
  echo("$ip is not a valid IP address");
}
?>
```

Sanitize and Validate an Email Address

The following example uses the filter_var() function to first remove all illegal characters from the $email variable, then check if it is a valid email address:

Example

```
<?php
$email = "john.doe@example.com";

// Remove all illegal characters from email
$email = filter_var($email, FILTER_SANITIZE_EMAIL);

// Validate e-mail
if (!filter_var($email, FILTER_VALIDATE_EMAIL) === false)

{
  echo("$email is a valid email address");
}

else

{
  echo("$email is not a valid email address");
```

```
}
?>
```

Sanitize and Validate a URL

The following example uses the filter_var() function to first remove all illegal characters from a URL, then check if $url is a valid URL:

Example

```
<?php
$url = "https://www.google.com";

// Remove all illegal characters from a url
$url = filter_var($url, FILTER_SANITIZE_URL);

// Validate url
if (!filter_var($url, FILTER_VALIDATE_URL) === false)
{
  echo("$url is a valid URL");
}
else
{
  echo("$url is not a valid URL");
}
?>
```